

Market Value Index / MVA System Analysis

Generated from repository inspection of the Crypto_AML workspace.

1. Purpose of MVRV and What It Is

MVRV is a crypto analytics concept that compares market value to realized value. In general, it is used to identify when an asset is overvalued or undervalued by comparing the current market capitalization to the value at which coins were last moved on-chain.

In this system, the implemented feature is called Market Value Index (MVA), not a classical MVRV ratio. The system focuses on portfolio-level USD book value for wallets and clusters, rather than directly computing market-cap-to-realized-value.

Key system purposes:

- Rank blockchain addresses and clusters by current USD balance exposure.

- Provide portfolio intelligence for identified addresses or clusters using on-chain token holdings.

- Show asset allocation mixes across ETH, stablecoins, other ERC-20 tokens, and DeFi exposure.

- Surface intelligence flags for high-value, stablecoin-heavy, dormant, and ETH-dominant holdings.

2. Implementation in This System

The Market Value Index implementation spans both frontend and backend code. The main pieces are:

Frontend list: `crypto-aml-tracker/src/pages/MarketValueIndex.jsx`

Frontend address/cluster analysis:

`crypto-aml-tracker/src/components/intelligence/MarketValueAnalysis.jsx`

API routes: `crypto-aml-tracker/backend-py/routes/mva.py`

Balance aggregation service: `crypto-aml-tracker/backend-py/services/balance_aggregator.py`

What each frontend column means in the Market Value Index table:

Address: Ethereum address from the AML graph or pipeline tables.

Book USD: current cached USD value of all tracked token balances for that address.

ETH + WETH (Ξ): sum of ETH and wrapped ETH token units held, not USD value.

Native wei (ETH): raw native ETH balance in wei, shown compactly.

Stables USD: USD value of stablecoin balances recognized by the system.

Synced: timestamp when the last balance update was recorded.

Stablecoin treatment:

The backend treats USDT, USDC, DAI, and BUSD as stablecoins. Those balances are aggregated into `stablecoin_value_usd` and used to calculate a stablecoin allocation percentage.

These tokens are translated into USD using CoinGecko price lookups and stored in the `wallet_balances` table. That USD value then appears in the MVA table and charts.

Backend data translation and aggregation flow:

Fetch on-chain balances via Ethereum RPC for ETH and selected ERC-20 tokens.

Fetch USD prices from CoinGecko for ETH, WETH, USDT, USDC, and DAI (cached for 5 minutes).

Save per-address balances into `wallet_balances` with `balance_usd` values.

Aggregate wallet balances to compute `total_value_usd`, `eth_value_usd`, `stablecoin_value_usd`, and token counts.

For clusters, aggregate balances across all cluster member addresses into `cluster_portfolios`.

Persist snapshots into `portfolio_history` for historical charts.

Important implementation details:

The list endpoint `/api/mva/directory` merges addresses from `addresses`, `wallet_balances`, `placement_entity_addresses`, and `layering_entity_addresses`, then joins cached balance data.

If a wallet has no cached balances yet, it still appears in the index with zeros until it is refreshed.

The address analysis endpoint `/api/mva/address/{address}` refreshes balances if the data is missing or older than one hour.

The cluster analysis endpoint `/api/mva/cluster/{cluster_id}` refreshes cluster balances when stale and returns aggregated allocation and assets.

Historical values come from `portfolio_history` and are returned by `/api/mva/history/{entity_id}`.

3. Presentation Strategy for Higher-Ups

When presenting this system, keep the message concise and business-focused. Higher-ups want to understand value, risk signals, and how the dashboard supports AML decision-making.

Lead with the objective: this system ranks suspicious blockchain entities by current USD exposure and shows portfolio composition in ETH, stablecoins and other crypto holdings.

Explain the data source: on-chain balances are fetched for Ethereum native ETH and major ERC-20 tokens, then converted to USD via market prices.

Demonstrate the core view: Market Value Index table, which surfaces high-value addresses and enables quick workspace drilling.

Highlight the analysis view: portfolio summary cards, holding breakdown, historical trend, and allocation pie chart.

Point out risk flags: high-value, stablecoin concentration, dormant holdings, and ETH dominance.

Suggested presentation structure:

- 1) Problem and opportunity: AML investigators need fast visibility on wallet value and stablecoin exposure.
- 2) What the system shows: ranking, composition, history, and risk flags.
- 3) What is currently automated: cached MVA refresh on demand and hourly stale detection.
- 4) What is missing: true live streaming, wider token coverage, and classical MVRV ratio.
- 5) Recommended next steps: add live updates, more token types, better asset-class labeling, and portfolio trend alerts.

4. Graphs and Pie Charts: What They Show and Whether They Work

The MVA user interface includes two primary visualizations:

Timeline chart: shows USD portfolio value over time, with separate lines for ETH value and stablecoin value.

Allocation pie chart: shows the current percentage mix of ETH, stablecoins, other ERC-20 tokens, and DeFi exposure.

Does it work correctly?

Yes, the chart logic is implemented correctly for the data returned by the API, including a fallback flat line when history is sparse.

The timeline chart uses `ComposedChart` and plots total USD, ETH USD, and stablecoin USD. The lines are color-coded and tooltip labels are accurate.

The pie chart is generated from allocation percentages computed in the backend. It renders non-zero segments and falls back to a placeholder if no allocation data exists.

Current limitations to be aware of:

The backend currently sets `defi_percent` to 0 for address-level analysis, so the pie chart does not accurately represent DeFi exposure for single addresses.

Stablecoins are limited to USDT, USDC, DAI, and BUSD. Other USD-like tokens are not included in `stablecoin_value_usd`.

Portfolio history is only available if snapshots were recorded; if history is missing, the chart shows a synthetic flat interval.

There is no true live update mechanism; charts refresh only on page load and when the user clicks Refresh.

5. Recommendations to Improve the System

Here are practical enhancements for the analytics, data coverage, visualization, and updates.

Add a true MVRV ratio calculation if the business wants the standard market-value-to-realized-value signal; this requires tracking realized on-chain cost basis or historical entry values.

Expand token coverage beyond ETH, WETH, USDT, USDC, DAI, and BUSD to include more major ERC-20 holdings and on-chain stablecoins.

Implement DeFi category detection for tokens that represent lending, staking, or liquidity positions so allocation and risk flags become more meaningful.

Add an auto-refresh mechanism on the frontend with a short poll interval (for example 30-60 seconds) and a backend endpoint that can return only changed addresses or portfolios.

For best real-time behavior, add WebSocket or server-sent events so the API can push portfolio updates when balance snapshots are written.

Add a visible "last refreshed" timestamp for portfolio and history views, so investigators know how fresh the numbers are.

Enhance the Main Index table with a high/low value delta, stablecoin ratio, and alert badge for stale or missing balance data.

Add a dashboard summary card for total on-book value, stablecoin share, and number of high-value wallets to make executive reporting easier.

Consider export options: PDF report export, CSV download, or slide-ready summaries for higher-ups.

If live update is required, the backend should capture price changes and address balance changes separately, then send incremental updates rather than reloading full portfolios every time.

Live update implementation notes:

The current frontend fetches `/api/mva/directory`, `/api/mva/address/{address}`, and `/api/mva/history/{entity_id}` only on demand or initial render.

To implement live updates, add a background refresh timer in the frontend and/or a websocket connection from a new backend socket service.

The backend can publish events when `wallet_balances` or `portfolio_history` changes, and the frontend can consume those events to refresh only affected rows.

If WebSocket is not possible, use polling with an endpoint such as `/api/mva/updates?since={timestamp}`.

Report generated from repository source files and runtime route implementation analysis.